

UNIVERSITÀ DEGLI STUDI DI SALERNO



**Dipartimento di Ingegneria dell'Informazione ed
Elettrica e Matematica applicata**

Corso di Laurea in Ingegneria Informatica

ELABORATO DI LAUREA

Tecniche di descrizione dell'immagine

Relatore:
Ch. Prof. Nicola Capuano

Candidato
Ciro Ronca
Matricola: 0612708051

Anno Accademico 2024/2025

Sommario

Introduzione	5
Capitolo 1: Ambiente di sviluppo	6
1.1 Sviluppo dell'applicazione	6
1.2 Interfaccia Utente.....	6
1.3 Tecnologie per la descrizione delle immagini.....	7
1.4 Controlli	7
1.5 Interfaccia utente personalizzata	8
1.6 Database	8
1.7 Tecnologie Scartate.....	8
Capitolo 2: Progettazione della soluzione	9
2.1 Swagger UI.....	13
2.2 Microsoft Azure Computer Vision API.....	15
2.2.1 Primi Test	15
2.3 Groq API.....	16
2.3.1 Test con Groq API	16
2.3.2 Blip API.....	17
2.4 Clarifai API	17
2.4.1 Test con Clarifai.....	18
2.5 Risultato Finale.....	18
2.6 Limitazioni del sistema iniziale.....	18
2.6.1 Dataset.....	19
2.6.2 WikiArt.....	19
2.7 Nuova prospettiva.....	19
Capitolo 3: Interazione con il sistema	21
3.1 Chat in tempo reale con Groq (Blazor).....	21
3.2 Textfield	21
3.3 Diagramma di Sequenza	22
Capitolo 4: Funzionalità Aggiuntive	25
4.1 Controllo sull'immagine	25
4.2 Controllo Textfield	25
4.3 Interfaccia utente personalizzata	26
4.4 Database	30
Capitolo 5: Implementazione e descrizione delle funzionalità	32

5.1 Prompt di Groq.....	32
5.1.1 Prima versione	32
5.1.2 Seconda versione	33
5.1.3 Versione definitiva.....	33
5.2 Creazione Memoria Groq.....	35
5.2.1 Metodi Creati.....	35
Capitolo 6: Demo	37
Capitolo 7: Conclusioni e prospettive future	40

Introduzione

L'Intelligenza Artificiale è la disciplina che studia tecniche e metodi volte a una macchina (sistema intelligente), che ha lo scopo di risolvere problemi complessi, simulando alcune capacità dell'essere umano, come il ragionamento, l'apprendimento e la percezione. Negli ultimi anni sta prendendo sempre più piede, grazie ai rapidi progressi, la capacità di catturare un'immagine e farla descrivere in maniera del tutto automatica a un sistema intelligente, al fine di estrarre e fornire informazioni all'essere umano, o ad altri sistemi. Questa tematica, nota come *image description* si colloca all'interno del campo della *Computer Vision*, ad oggi oggetto di numerosi studi e applicazioni pratiche.

Nel nostro quotidiano queste tecnologie sono molto rilevanti, poiché permettono di superare barriere come la disabilità visiva, offrendo la possibilità di trasformare un contenuto visivo in una rappresentazione testuale. Ciò consente di ampliare l'accessibilità e migliorare **l'interazione uomo-macchina**. Altre applicazioni riguardano il settore della sicurezza, grazie all'utilizzo di sistemi di videosorveglianza intelligente e l'ambito della diagnostica medica, dove la descrizione automatica delle immagini può supportare l'analisi clinica.

A tal fine, è stato sviluppato un sistema intelligente in grado di descrivere delle immagini inserite dall'utente, consentendo **un'interazione diretta con il sistema stesso**. Inoltre, viene data una particolare attenzione alla qualità dell'output generato, cercando di garantire che le descrizioni fossero coerenti con il contenuto dell'immagine, corrette dal punto di vista informativo ed efficienti nell'elaborazione.

All'interno della tesi sono presenti diversi capitoli e nello specifico, vedremo nel Capitolo 1 l'ambiente di sviluppo del progetto. Successivamente nel Capitolo 2 verrà illustrata la progettazione della soluzione per poi proseguire con il Capitolo 3 dove verrà spiegato come l'utente interagisce, in maniera significativa, con la macchina. Il Capitolo 4 illustrerà delle funzionalità aggiuntive e il Capitolo 5 presenterà l'implementazione e descrizione delle funzionalità. In conclusione, abbiamo il Capitolo 6 dove viene mostrata una piccola demo e l'ultimo Capitolo, il sesto, dove avremo le conclusioni e prospettive future.

Capitolo 1: Ambiente di sviluppo

In questo capitolo viene presentata una panoramica delle tecnologie utilizzate per lo sviluppo del sistema. Verranno inoltre illustrate le tecnologie scartate, spiegando nei Capitoli 2 e 3 le motivazioni che hanno portato alla loro esclusione

1.1 Sviluppo dell'applicazione

È stato scelto **Visual Studio 2022** come ambiente di sviluppo, con utilizzo del framework ASP.NET Core (.NET 8). Il linguaggio di programmazione, invece, è **C#**, poiché particolarmente adatto per lo sviluppo di applicazioni web basate su servizi e API.

1.2 Interfaccia Utente

Per l'Interfaccia Utente **iniziale**, per poi cambiarla con una **UI personalizzata**, è stato utilizzato **Swagger UI**, che consente di far funzionare e testare in modo interattivo le diverse **API**¹ disponibili su internet. Swagger consente a chiunque, dal gruppo di sviluppo ai consumatori finali, di visualizzare e interagire con le risorse dell'API senza dover implementare alcuna logica.

L'interfaccia iniziale è molto essenziale ma presenta già alcuni elementi importanti, come il bottone “*Try it out*” seguito poi dal bottone “Execute” che permette di provare le chiamate direttamente dal browser, senza bisogno di strumenti esterni, e diverse API colorate in base al metodo **HTTP**² (GET, POST, PUT, DELETE). È stato deciso di utilizzare una chiamata di tipo **POST**³ per inviare le immagini al server. Infine, grazie a Swagger UI, non è necessario creare un box separato per visualizzare le risposte, perché l'interfaccia lo mostra automaticamente.

¹ Le **API** (*Application Programming Interface*) sono un insieme di regole e strumenti che permettono a due programmi di comunicare tra di loro, queste sono state di fondamentale importanza poiché tutto il progetto si basa sull'utilizzo di API.

² **HTTP** (**H**yper**T**ext **T**ransfer **P**rotocol) è un protocollo di comunicazione utilizzato per lo scambio di informazioni tra client e server sul web. Permette di inviare richieste (request) e ricevere risposte (response), supportando diversi metodi come GET, POST, PUT e DELETE.

³ La chiamata **POST** è utilizzata quando è necessario inviare dati al server, come file o informazioni nel body della richiesta. Al contrario, **GET** serve principalmente a richiedere dati già presenti sul server.

1.3 Tecnologie per la descrizione delle immagini

- **Microsoft Azure Computer Vision API:** Ovvero una piattaforma di **cloud computing**⁴ di Microsoft, che offre servizi per creare, eseguire e gestire applicazioni e dati tramite server distribuiti in tutto il mondo. Tra i servizi proposti è stato utilizzato Microsoft Azure Computer Vision API, ovvero un servizio di intelligenza artificiale fornito da Microsoft Azure che permette di analizzare immagini e dare una breve descrizione.
- **Groq API:** un servizio che permette di sfruttare modelli di intelligenza artificiale di tipo *Large Language Models*⁵ per generare, riassumere, tradurre o arricchire testi, utilizzando come modello “*meta-llama/llama-4-scout-17b-16e-instruct*”
- **Clarifai API:** ovvero un servizio di intelligenza artificiale specializzato nell’analisi di immagini e video. Permette di identificare e classificare oggetti, animali, persone, scene, colori e altre caratteristiche visive presenti nei contenuti multimediali. Grazie a **modelli pre-addestrati**, Clarifai è in grado di fornire **tag** dettagliati e informazioni strutturate, facilitando l’elaborazione automatica delle immagini e l’integrazione in applicazioni che richiedono riconoscimento visivo, analisi dei contenuti o supporto decisionale basato su dati visivi.
- **TextField:** componente centrale del progetto, situato all’interno dell’interfaccia grafica, che permette all’utente di inserire un messaggio di testo che viene utilizzato poi come input al sistema intelligente. L’importanza di questo elemento sarà approfondita nel Capitolo 3, Paragrafo 3.3.

1.4 Controlli

Per fornire un’esperienza migliore all’utente, sono stati introdotti diversi **controlli** che vanno a migliorare il progetto creato, rendendolo più affidabile e fluido:

- **Controllo immagine:** garantisce che l’utente inserisca effettivamente un file e che esso sia solo ed esclusivamente un file immagine.
- **Controlli di qualità dell’immagine inserita:** vengono effettuati dei controlli sulla qualità dell’immagine, scartando ad esempio immagini troppo sfocate o con troppa luminosità.

⁴ **Cloud computing :**

Il cloud computing indica un’erogazione di servizi offerti su richiesta da un fornitore a un utente finale attraverso la rete internet (come l’archiviazione, l’elaborazione o la trasmissione dati), a partire da un insieme di risorse preesistenti, configurabili e disponibili in remoto sotto forma di architettura distribuita

⁵ I **Large Language Models (LLM)** sono modelli di intelligenza artificiale addestrati su grandi quantità di testi. Grazie a questo addestramento, sono in grado di comprendere, generare e rielaborare linguaggio naturale in modo coerente e contestuale. Vengono utilizzati per compiti come la generazione di testo, la traduzione automatica, la risposta a domande e il riassunto di contestuale

- **Controllo Textfield:** viene controllato che l'utente non inserisca messaggi vuoti, i quali possano causare errori nell'output generato.

1.5 Interfaccia utente personalizzata

Un'interfaccia utente personalizzata, realizzata tramite **HTML, CSS e JavaScript**, in modo tale da offrire un'esperienza migliore all'utente, rendendo la schermata più semplice e intuitiva, rispetto a Swagger UI, integrando inoltre nuove tecnologie e funzionalità.

1.6 Database

Il **Database** è nient'altro che una collezione di dati strutturati, connessi logicamente tra di loro. Inserendolo, viene dato all'utente di visualizzare nella propria UI, tutte le immagini inserite seguite dalla descrizione fornita dal sistema intelligente. La scelta del database è ricaduta su **SQLite**, per motivi spiegati successivamente.

1.7 Tecnologie Scartate

- **Blip API:** Esso si basa sul modello BLIP, sviluppato da *Salesforce Research* per collegare testo e immagini. Il modello in questione è in grado di generare descrizioni automatiche delle immagini.
- **Dataset:** Un dataset è una collezione strutturata di dati, generalmente di grandi dimensioni e organizzata secondo un dato modello relazionale. Nel caso specifico, un dataset costituisce la **base di conoscenza** su cui un sistema di apprendimento automatico si allena per acquisire capacità di riconoscimento e generalizzazione.
- **Chat in tempo reale con Blazor:** Microsoft offre una tecnologia basata su .NET (stesso framework che l'azienda ha fatto usare), che consente di sviluppare interfacce web interattive utilizzando C#, ovvero **Blazor**. Blazor supporta due modelli principali di esecuzione: Blazor WebAssembly, dove il codice C# viene compilato in WebAssembly ed eseguito nel browser, e Blazor Server, dove l'applicazione gira sul server e comunica con il browser tramite SignalR⁶.

⁶ **SignalR:** È una libreria di **ASP.NET** sviluppata da Microsoft che permette la **comunicazione in tempo reale** tra server e client web.

Capitolo 2: Progettazione della soluzione

In questo capitolo si illustra il processo di ideazione e progettazione della soluzione sviluppata. Vengono inoltre analizzate le motivazioni che hanno portato a scegliere una tecnologia al posto di un'altra, in funzione degli obiettivi del progetto.

Inoltre, è stata presa in considerazione la realizzazione di uno strumento fondamentale nella modellazione **UML**⁷, ovvero i **casi d'uso**. Un Caso d'Uso (*Use case*) rappresenta un insieme di scenari di funzionamento del sistema collegati tra loro per il fatto di essere finalizzati al raggiungimento di uno specifico obiettivo dell'utente. Essi descrivono **come gli attori (utenti) interagiscono con il sistema** per raggiungere determinati obiettivi, evidenziando flussi principali e alternativi di eventi, nonché eventuali pre- e post-condizioni.

Inizialmente, quindi viene fatta una **descrizione dei casi d'uso**, mentre successivamente, in **Figura 1**, viene rappresentato un **Diagramma UML**, ovvero il **diagramma dei casi d'uso (Use Case Diagram)**, che di fatto permette di **visualizzare** le funzionalità principali del sistema dal punto di vista degli attori, evidenziando le interazioni tra attori e casi d'uso.

Ecco come si presentano le descrizioni e il diagramma dei casi d'uso:

1. Caricare immagine

- **Attore partecipante:** User.
- **Pre-condizioni:**
 1. L'utente avvia il programma correttamente.
 2. L'utente visualizza l'interfaccia Swagger UI.
- **Post-condizioni:** L'immagine è caricata e disponibile per le analisi.
- **Flusso di eventi:**
 1. L'utente seleziona il comando per inserire l'immagine.
 2. L'utente sceglie un file immagine dal proprio dispositivo.
 3. L'immagine viene caricata nel sistema.
- **Flusso di eventi alternativi:**

2a: L'utente seleziona un file non valido, di conseguenza viene mostrato un messaggio di errore.

⁷ In **ingegneria del software**, **UML** (*Unified Modeling Language*, "linguaggio di modellizzazione unificato") è un linguaggio di modellazione e di specifica basato sul paradigma orientato agli oggetti.

2. Inserire messaggio utente

- **Attore partecipante:** User.
- **Pre-condizioni:**
 1. L'utente avvia il programma correttamente.
 2. L'utente visualizza l'interfaccia Swagger UI.
- **Post-condizioni:**
 1. Il messaggio viene utilizzato da Groq.
 2. Il messaggio viene memorizzato dal sistema.
 3. L'utente visualizza il messaggio inserito.
- **Flusso di eventi:**
 1. L'utente inserisce un messaggio di testo.
 2. Il sistema acquisisce il contenuto.
 3. Groq produce la descrizione tenendo conto del messaggio dell'utente.
 4. Il sistema salva il messaggio dell'utente.
 5. Il sistema mostra il messaggio dell'utente.
- **Flusso di eventi alternativi:**
 - 1a: L'utente non inserisce alcun testo, quindi il sistema considera solo i risultati delle analisi automatiche.
 - 1a.1: L'utente inserisce un messaggio non valido e il sistema invita a scrivere un altro messaggio.
 - 3a: Groq non risponde, quindi il sistema mostra i soli risultati di Azure/Clarifai.

3. Visualizzare output

- **Attore partecipante:** User.
- **Pre-condizioni:**
 1. L'utente avvia il programma correttamente.
 2. L'utente visualizza l'interfaccia Swagger UI.
 3. Sono stati caricati immagine e messaggio utente corretti.
- **Post-condizioni:** L'utente visualizza una descrizione, i risultati delle API e il messaggio utente inserito.
- **Flusso di eventi:**
 1. Il sistema ottiene analisi da Azure.
 2. Il sistema ottiene tag da Clarifai.
 3. Il sistema ottiene il messaggio utente.
 4. Il sistema arricchisce la descrizione con Groq.
 5. L'utente visualizza l'output finale.
- **Flusso di eventi alternativi:**
 - 1.a: Azure non è raggiungibile, il sistema segnala l'errore e continua con altri servizi.
 - 2.a: Clarifai non è disponibile, il sistema segnala l'errore e continua con altri servizi.

3.a: L'utente non inserisce alcun messaggio.

4.a: Groq non risponde, quindi il sistema mostra i soli risultati di Azure/Clarifai.

4. Richiedere analisi ad Azure

- **Attori partecipanti:** Sistema, Azure
- **Pre-condizioni:** È disponibile un'immagine caricata.
- **Post-condizioni:** Il sistema riceve i risultati di Azure.
- **Flusso di eventi:**
 1. Il sistema invia l'immagine ad Azure.
 2. Azure restituisce i dati di analisi (descrizione, colori, oggetti rilevati).
- **Flusso di eventi alternativi:**

1a: Azure non è raggiungibile, il sistema segnala l'errore e continua con altri servizi.

5. Ottenere tag da Clarifai

- **Attori partecipanti:** Sistema, Clarifai
- **Pre-condizioni:** È disponibile un'immagine caricata.
- **Post-condizioni:** Il sistema riceve i tag da Clarifai.
- **Flusso di eventi:**
 1. Il sistema invia l'immagine a Clarifai.
 2. Clarifai restituisce i tag rilevati.
- **Flussi di eventi alternativi:**

1a: Clarifai non è disponibile, il sistema segnala l'errore e continua con altri servizi.

6. Arricchire descrizione con Groq

- **Attori partecipanti:** Sistema, Groq
- **Pre-condizioni:** Sono disponibili i risultati preliminari (Azure/Clarifai) e l'eventuale messaggio utente.
- **Post-condizioni:** Il sistema ottiene una descrizione arricchita.
- **Flusso di eventi:**
 1. Il sistema invia descrizioni/tag e messaggio utente a Groq.
 2. Groq genera una descrizione testuale migliorata.
- **Flusso di eventi alternativi:**

2a: Groq non risponde, quindi il sistema mostra i soli risultati di Azure/Clarifai.

7. User Message

- **Attori partecipanti:** User, Sistema
- **Pre-condizioni:** L'utente ha inserito un messaggio.
- **Post-condizioni:** Il messaggio viene incluso nell'arricchimento finale.
- **Flusso di eventi :**
 1. L'utente scrive un messaggio di testo.
 2. Il sistema utilizza il contenuto per personalizzare l'output.
- **Flussi do eventi alternativi:**

1a: Nessun messaggio inserito, quindi il sistema prosegue solo con i dati di Azure/Clarifai.

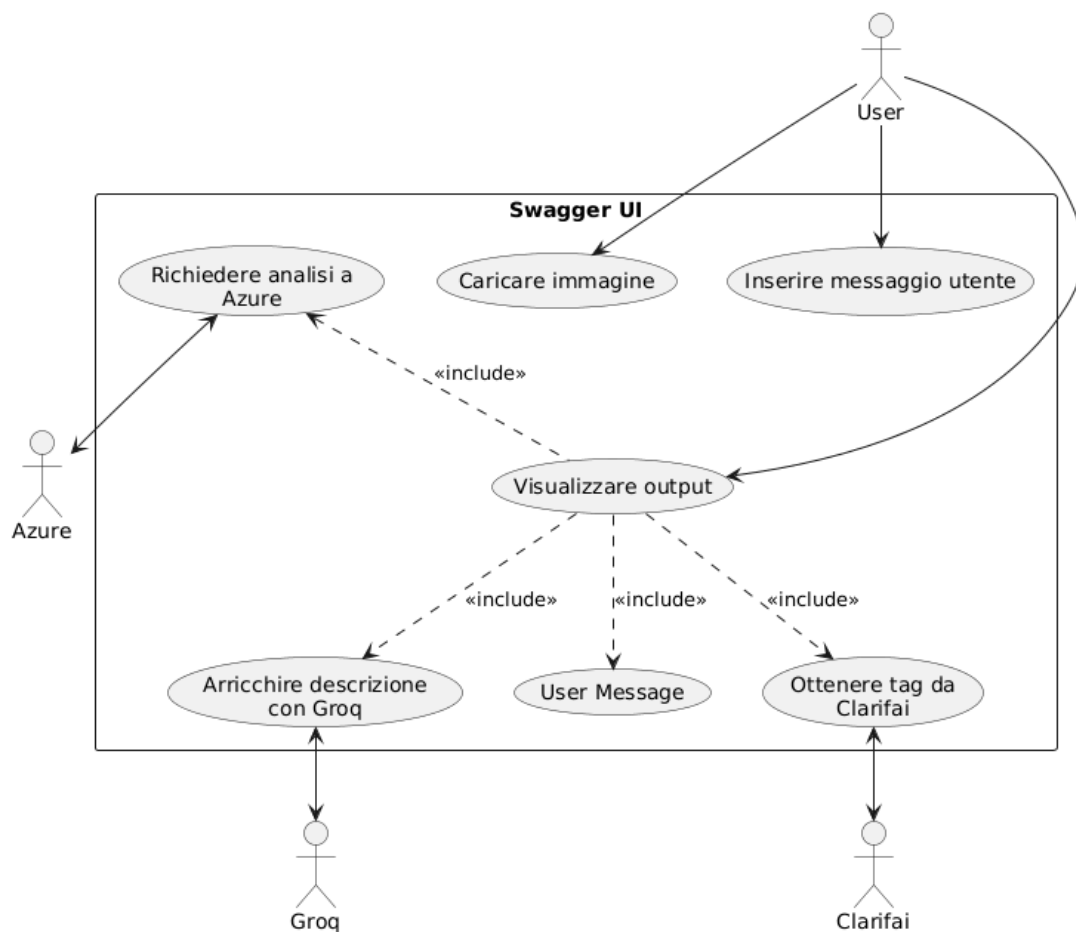


Figura 1- Diagramma dei casi d'uso

La simbologia utilizzata è la seguente:

- **Attore:** rappresenta un utente e un sistema esterno che interagisce con il sistema, raffigurato con un omino stilizzato.
- **Caso d'uso:** rappresenta una funzionalità del sistema, raffigurato con un'ellisse.
- **Include (<<include>>):** indica che un caso d'uso include sempre un altro caso d'uso, ossia quest'ultimo viene eseguito ogni volta che il primo viene invocato.
- **Box (Swagger UI):** rappresenta un'area di lavoro o un contenitore visivo all'interno della quale sono organizzati e delimitati gli elementi del diagramma, facilitando la lettura e la comprensione delle interazioni.

2.1 Swagger UI

La prima sfida progettuale ha riguardato la realizzazione di un'interfaccia utente. Quest'ultima deve permettere il caricamento del file di immagine e la visualizzazione dell'output generato dal sistema. A questo scopo è stato adottato **Swagger UI**.

All'interfaccia manca la possibilità di aggiungere immagini; per superare questa limitazione è stato introdotto un campo dove si potesse inserire l'URL⁸ dell'immagine. Tale soluzione si è rivelata poco pratica ed è stata sostituita con la possibilità di selezionare direttamente un file dal computer dell'utente, semplificando l'interazione.

Successivamente, viene mostrato il diagramma delle classi riportato in **Figura 2**, dove mostra la struttura interna del sistema, **evidenziando le principali classi** coinvolte nell'elaborazione delle immagini e la loro interazione con le API esterne (Azure, Clarifai e Groq). L'obiettivo è fornire una rappresentazione chiara delle responsabilità di ciascuna classe e dei metodi principali esposti, facilitando la comprensione dell'architettura software dal lato backend⁹.

⁸ **Uniform Resource Locator (URL):** è una sequenza di caratteri che identifica in modo univoco l'indirizzo di una risorsa su una rete di computer, come un documento, un'immagine o un video, tipicamente presente su un host server e resa accessibile tramite un client

⁹ Il termine **backend** indica la parte di un sistema informatico o di un'applicazione che opera "dietro le quinte", gestendo logica, dati e elaborazioni necessarie a supportare le funzionalità visibili all'utente.

La simbologia utilizzata nel diagramma delle classi è la seguente:

- **Classe:** rappresenta un insieme di oggetti con attributi e metodi comuni, raffigurata con un rettangolo suddiviso in tre sezioni: nome della classe, attributi e metodi.
- **Attributo:** rappresenta una proprietà della classe. Si indica con un simbolo di visibilità (+ pubblico, - privato) seguito dal nome e, opzionalmente, dal tipo.
- **Metodo:** rappresenta una funzionalità della classe. Si indica con un simbolo di visibilità (+ pubblico, - privato) seguito dal nome e dalla lista dei parametri, con il tipo di ritorno.

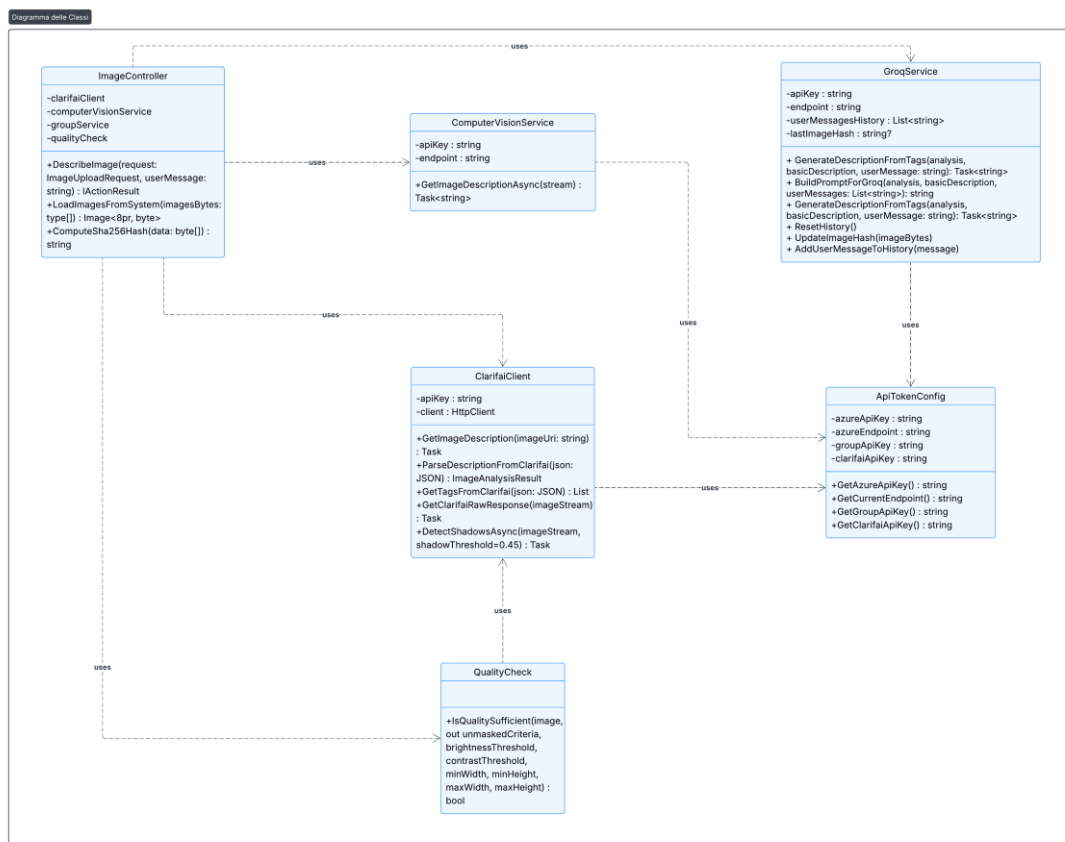


Figura 2 - Diagramma delle Classi

Successivamente, una breve descrizione dei compiti principali che svolgono le classi:

1. **ImageController:** La classe **ImageController** riceve le immagini e il messaggio scritto dall'utente, converte i byte dell'immagine in un oggetto `Image<Bgr, byte>` e calcola l'**hash SHA-256** dei byte dell'immagine per identificare quell'immagine.

2. **ApiTokenConfig**: in questa classe sono salvate i token che permettono di collegare il progetto ai **servizi esterni**.
3. **Servizi esterni (QualityCheck, ClarifaiClient, ComputerVisionService, GroqService)**
 - **QualityCheck**: eventuali controlli sulla qualità e dimensione delle immagini.
 - **ClarifaiClient**: interagisce con API Clarifai per estrazione tag e colori utili all'elaborazione e ai controlli di qualità.
 - **ComputerVisionService**: interagisce con API Azure Computer Vision per descrizione immagine.
 - **GroqService**: interagisce con API Groq per generare descrizioni avanzate. Inoltre, si occupa di gestire la memoria dei messaggi dell'utente e di generare la descrizione una volta ricevuto in input le diverse informazioni.

In seguito, è stata implementata un'API esterna in grado di ricevere l'immagine e restituirne una descrizione.

2.2 Microsoft Azure Computer Vision API

Azure API è stata scelta grazie alla sua **affidabilità**, essendo un servizio Microsoft, e dalla facilità di integrazione con il progetto sviluppato in C#. Per utilizzare l'API, è stato necessario creare una risorsa dedicata ad AI Vision su Azure e generare un token¹⁰ di accesso, informazioni che sono state poi inserite in una specifica classe all'interno del progetto.

2.2.1 Primi Test

Le immagini utilizzate sono state selezionate seguendo una strategia ben precisa: **si è iniziato con immagini semplici, contenenti pochi dettagli, per poi passare a immagini via via più complesse**, in modo da valutare come il sistema fosse in grado di estrapolare informazioni di diversa difficoltà. Si è scelto una foto di una matita su sfondo nero, per poi testare immagini più complesse, come tramonti e paesaggi, fino a opere d'arte famose quali *La Notte Stellata di Van Gogh* e *La Nascita Di Venere*.

Le descrizioni fornite dall'API risultavano **troppo minimali**: questo approccio era accettabile solo per immagini semplici, mentre per immagini con un maggior numero di dettagli la descrizione rimaneva comunque molto generica. Ad esempio, caricando

¹⁰ **Token**: un codice di autorizzazione fornito da un servizio o API che permette a un'applicazione di accedere in modo sicuro alle risorse protette, garantendo che solo utenti o sistemi autorizzati possano effettuare richieste.

l'immagine de *La Nascita di Venere* di Botticelli, la descrizione restituita era semplicemente: “*arte di un gruppo di persone*”.

Per immagini complesse, è stato necessario valutare alternative o soluzioni in grado di ampliare la descrizione generata da Azure API.

Inizialmente, è stata considerata l'adozione di API in grado di fornire descrizioni più dettagliate rispetto a Microsoft Azure; tuttavia, utilizzando esclusivamente API gratuite, questo rappresentava il limite massimo delle prestazioni ottenibili. Tra le alternative considerate rientrava anche l'API di **OpenAI** il cui utilizzo prevedeva costi legati a ogni richiesta, rendendo questa opzione non praticabile nel contesto del progetto.

2.3 Groq API

Successivamente, è stata individuata un'API, che fosse in grado di prendere in input il testo generato da Microsoft Azure e di arricchirlo, ampliandolo e rendendolo più dettagliato. Si tratta di **Groq API**.

Si è optato per Groq poiché dà la possibilità di fornire istruzioni ben precise sul risultato desiderato. Questa operazione viene effettuata tramite l'uso di un *prompt*, ovvero un testo di input in cui vengono inserite indicazioni dettagliate che guidano il modello nella generazione dei contenuti (nel Capitolo 5 verrà mostrato il contenuto del *prompt*). In questo modo è possibile avere un maggiore controllo sul tipo di informazioni che si desidera vengano incluse nella descrizione finale.

Per l'integrazione è stato necessario generare un **token di accesso** direttamente dal portale Groq, il quale è stato utilizzato all'interno del codice per autenticare le richieste all'API. Una volta autenticati, è stato possibile inviare i testi generati da Microsoft Azure e ricevere in risposta descrizioni più elaborate e coerenti con il contenuto delle immagini.

Una volta impostato il prompt e collegato l'input testuale generato da Microsoft Azure, sono stati eseguiti nuovamente i test.

2.3.1 Test con Groq API

Le immagini utilizzate sono sempre le stesse del test precedente, in modo da mantenere un confronto coerente, ovvero, si parte da un'immagine scarsa di dettagli fino a un'immagine piena di dettagli.

I primi test hanno mostrato che, nonostante Groq fosse in grado di ampliare il testo ricevuto, l'output generato rimaneva talvolta generico o impreciso. Questo era dovuto principalmente alla qualità dell'input fornito: se il testo di Microsoft Azure era troppo sintetico e limitato nei dettagli, anche Groq produceva **descrizioni limitate, poco accurate o addirittura si inventava delle informazioni non presenti nell'immagine.**

Questi risultati evidenziano due aspetti rilevanti: il primo è che l'accuratezza di Groq dipende fortemente dalla qualità dell'input iniziale; un input povero di informazioni limita significativamente l'efficacia dell'output. Il secondo è che la possibilità di fornire manualmente un messaggio tramite prompt a Groq migliora sensibilmente la qualità delle descrizioni generate, consentendo un maggiore controllo sulla fase di input.

Sulla base di queste osservazioni, sono state condotte ulteriori ricerche e sperimentazioni, volte a ottimizzare la quantità e la qualità delle informazioni passate a Groq, al fine di ottenere descrizioni quanto più dettagliate, complete e veritiere possibile.

2.3.2 Blip API

Inizialmente è stata individuata l'**API Blip**, che funzionava in modo simile ad Azure: permetteva di analizzare immagini e generare descrizioni testuali. È stata integrata nel codice e testata: i risultati erano soddisfacenti e, combinata con Groq, il sistema era in grado di generare descrizioni più complete.

Tuttavia, durante i successivi test, l'API ha smesso di fornire un output. Dopo verifiche, si è constatato che la versione utilizzata era una versione di prova, e per continuare a usufruirne era necessario sottoscrivere un abbonamento a pagamento, analogamente a quanto accade con OpenAI. Di conseguenza, è stato necessario **rimuovere l'integrazione** e cercare una soluzione alternativa, senza compromettere il funzionamento complessivo del progetto.

2.4 Clarifai API

La soluzione alternativa prende il nome di **Clarifai API**. Per lo scopo del progetto, l'utilizzo di Clarifai ha rappresentato un significativo passo avanti, poiché consente di ottenere un'analisi più dettagliata delle immagini caricate dall'utente. In pratica, una volta che l'immagine viene inserita dall'utente, l'API elabora il contenuto visivo e genera un insieme di tag che descrivono gli elementi principali presenti, come i colori e oggetti.

2.4.1 Test con Clarifai

Durante i test, con la stessa regola dei precedenti, caricando un'immagine di un tramonto con vista su una città, Clarifai restituisce tag come [sole], [case], [tramonto], [arancione]. Questi tag rappresentano in maniera sintetica e strutturata ciò che è visibile nell'immagine, facilitando così la comprensione automatica del contenuto da parte del sistema.

Così facendo, adesso l'immagine viene descritta in maniera più completa rispetto a una singola frase generata da Azure, di fatto combinando entrambi gli output e dati in input a Groq, il sistema è in grado di fornire descrizioni più precise e dettagliate, migliorando sensibilmente la qualità delle informazioni restituite all'utente.

2.5 Risultato Finale

Il sistema è in grado di riconoscere monumenti storici come il Colosseo, generando una descrizione del tipo:

“In questa immagine, un imponente edificio antico domina la scena. La struttura, di colore marrone rossastro, presenta una serie di archi e colonne che si stagliano contro un cielo azzurro chiaro. Sullo sfondo, maestoso e imponente, si erge il Colosseo, simbolo dell'architettura e della storia della città. L'atmosfera è quella di un luogo carico di storia e cultura, con la luce del giorno che illumina la scena”

Come per tutti i test precedenti, anche in questa fase è stata rieseguita la serie completa di immagini selezionate per valutare le prestazioni del sistema. Di fatto riusciva a riconoscere senza problemi immagini semplici, come una matita su sfondo nero, descrivendola correttamente nei dettagli principali. Analogamente, le immagini di paesaggi e tramonti venivano identificate e descritte in maniera adeguata, cogliendo gli elementi chiave presenti nella scena.

Per quanto riguarda le opere d'arte più complesse e ricche di dettagli, il sistema non era in grado di riconoscerle singolarmente come specifici quadri famosi, ma riusciva comunque a fornire una rappresentazione generale e coerente della scena raffigurata. In questo modo, anche se mancava il riconoscimento preciso del nome o dell'autore, l'utente otteneva comunque una descrizione esaustiva e abbastanza informativa sul contenuto visivo dell'immagine.

2.6 Limitazioni del sistema iniziale

Il sistema intelligente sviluppato funzionava correttamente, tuttavia, si voleva che il sistema fosse in grado di riconoscere anche oggetti e opere d'arte famosi, poiché se non riesce a identificare elementi noti diventa un **limite significativo**: essendo un

sistema intelligente, si è ritenuto necessario che potesse fornire risposte complete e precise anche in questi casi, poiché se un utente, ad esempio, desidera la descrizione di un quadro, ci si aspetta che vengano forniti dettagli come il nome dell'opera, l'autore, lo stile e altre informazioni pertinenti, e il risultato ottenuto fino a questo punto non riesce a fornire queste informazioni.

2.6.1 Dataset

Per ovviare a questo problema si è pensato di costruire un vero e proprio **dataset**. Per utilizzare un dataset bisogna addestrarlo, quindi fare *training*. Con questo si intende l'utilizzo di un insieme di immagini già raccolte (e solitamente etichettate) per addestrare un modello di apprendimento automatico o di deep learning¹¹ a riconoscere pattern ricorrenti, caratteristiche visive e relazioni tra l'input (l'immagine) e l'output desiderato (la categoria o l'etichetta corrispondente)

2.6.2 WikiArt

Tra le possibili soluzioni è stato considerato **WikiArt**, ovvero, una enciclopedia, di circa 6GB di memoria, online dedicata all'arte, con l'obiettivo di rendere accessibile a tutti una vasta collezione di opere artistiche provenienti da diverse epoche, stili e movimenti, dove ogni quadro ha un'etichetta e una descrizione breve di essa.

Teoricamente, l'utilizzo del dataset WikiArt avrebbe potuto rappresentare una soluzione efficace: fornendo a Groq informazioni certe relative a un quadro (titolo, autore, stile), il modello sarebbe stato in grado di generare descrizioni più complete, arricchite da riferimenti storici e interpretazioni contestuali.

Tuttavia, l'approccio risultava applicabile solo al dominio delle opere pittoriche, senza coprire altri soggetti come sculture o monumenti. Inoltre, la realizzazione pratica avrebbe richiesto la costruzione di dataset estesi a una grande varietà di oggetti e luoghi famosi, con conseguente aumento della complessità di integrazione e addestramento. Considerati anche i vincoli temporali, si è ritenuto non perseguibile l'impiego di WikiArt nel progetto, pur riconoscendone il potenziale in scenari futuri di ricerca.

2.7 Nuova prospettiva

Dal tentativo precedente è emersa una criticità importante: il modello tratta tutte le informazioni ricevute **allo stesso modo**, senza distinguere tra **dati affidabili e dati**

¹¹ Il **deep learning** è un sottoinsieme del machine learning che si basa su reti neurali artificiali con molteplici strati di elaborazione, progettati per modellare rappresentazioni gerarchiche e astratte dei dati. Questa tecnica consente ai sistemi di apprendere automaticamente caratteristiche complesse a partire da grandi quantità di dati, risultando particolarmente efficace in ambiti come il riconoscimento vocale, la visione artificiale, l'elaborazione del linguaggio naturale e l'automazione di compiti complessi.

potenzialmente errati. Di conseguenza, anche informazioni imprecise possono influenzare la qualità delle descrizioni generate.

Le API di visione artificiale utilizzate, come Microsoft Azure e Clarifai, non offrono strumenti per filtrare automaticamente le informazioni dubbie, e Groq non è in grado di valutarne la correttezza. Questo significa che, senza un meccanismo di controllo, le descrizioni possono risultare parzialmente inaccurate o incomplete.

Per affrontare questo problema, il sistema è stato progettato per permettere **all'utente di intervenire direttamente.** In questo modo, l'utente può inserire informazioni certe, che verranno trattate in maniera diversa rispetto alle altre informazioni. Il risultato è un sistema più collaborativo, in cui l'utente e l'intelligenza artificiale lavorano insieme per ottenere risultati più affidabili e significativi.

Capitolo 3: Interazione con il sistema

L'interattività rappresenta, quindi, uno degli aspetti distintivi del sistema sviluppato: permette all'utente di influenzare direttamente le descrizioni generate dall'intelligenza artificiale, migliorandone precisione, completezza e coerenza.

In questo capitolo viene illustrato il meccanismo di **chat con Groq**, attraverso il quale l'utente può inserire input testuali. L'obiettivo è mostrare come l'interazione contribuisca a rendere il sistema più collaborativo e affidabile rispetto a un approccio puramente automatico

3.1 Chat in tempo reale con Groq (Blazor)

Un approccio efficace per realizzare l'interazione consiste nell'implementare una **chat in tempo reale** tra l'utente e Groq, simile a quella offerta da sistemi come ChatGPT o GeminAI.

L'utilizzo di Blazor è stato considerato per la realizzazione della chat, ma l'integrazione diretta si è rivelata complessa. Il progetto originale non era strutturato come **progetto Blazor**, rendendo necessaria una completa esportazione o ristrutturazione del progetto. Alla luce della complessità tecnica e dei vincoli temporali, questa soluzione non è stata implementata. Di conseguenza, è stata adottata una soluzione alternativa per consentire comunque l'interazione diretta dell'utente con Groq.

3.2 Textfield

La soluzione alternativa prevede l'utilizzo di un textfield, ovvero un piccolo spazio dove l'utente può inserire un messaggio e poi inviarlo a Groq quando si preme il pulsante "Execute".

Dopo aver implementato il textfield la risposta che dava Groq, una volta ricevuto il messaggio, era sempre precisa e coerente. La qualità delle descrizioni è stata migliorata grazie alla modifica del prompt fornito a Groq: il modello viene informato che il messaggio proviene dall'utente e che le informazioni inserite direttamente dall'utente, conoscitore dell'immagine, devono avere un peso maggiore rispetto agli altri dati.

Successivamente, per simulare l'esperienza di una chat, il prompt è stato aggiornato in modo da generare un messaggio fisso al termine di ogni descrizione, ovvero:

“La descrizione fornita è esaustiva? Se così non fosse, scrivimelo pure in un messaggio.”

Essendo una chat, il sistema intelligente integra un **meccanismo di memoria dei messaggi** (meccanismo che verrà spiegato nel Capitolo 5), che conserva tutte le interazioni precedenti dell'utente. Questo permette a Groq di mantenere coerenza tra le risposte successive e di utilizzare correttamente le informazioni fornite, evitando perdite di dati rilevanti. La memoria viene automaticamente azzerata quando l'immagine cambia, garantendo che le informazioni memorizzate siano sempre pertinenti al contenuto visualizzato. Inoltre, come verrà evidenziato nel capitolo successivo, sono stati introdotti controlli sul messaggio inserito dall'utente, al fine di prevenire errori derivanti da input non conformi.

Grazie a questo meccanismo, il sistema è in grado di generare descrizioni accurate e dettagliate per qualsiasi tipo di immagine. Di fatto, come verrà illustrato nel Capitolo 6, l'accuratezza delle descrizioni prodotte da Groq migliora progressivamente all'aumentare dei dettagli forniti dall'utente, evidenziando l'efficacia dell'interazione diretta nel perfezionamento dei contenuti generati

3.3 Diagramma di Sequenza

Per visualizzare in maniera visiva il flusso tra l'utente e il sistema intelligente sono stati realizzati due diagrammi di sequenza UML. Prima di mostrarli, è importante illustrare quali sono i componenti utilizzati:

-  Utente : **Attore esterno** che interagisce con il sistema.
-  SwaggerUI : **Boundary**, indica un'interfaccia del sistema che indica la parte con cui l'utente interagisce direttamente.
-  GroqService : **Control**, un servizio che gestisce la logica di controllo o il flusso delle operazioni.
-  GroqAPI **Entity**, un sistema esterno (API di Groq) che genera la descrizione dell'immagine a partire dai dati ricevuti.

Il primo diagramma viene raffigurato in **Figura 3** e mostra un esempio di cosa accade quando un utente inserisce un messaggio. In particolare, il diagramma evidenzia il flusso principale dell'interazione: il messaggio dell'utente viene ricevuto dall'interfaccia, passato al **controller**, memorizzato da **GroqService** e infine inviato alla **Groq API** per generare una descrizione arricchita. La risposta prodotta viene restituita all'utente tramite l'interfaccia, permettendo un'interazione diretta e coerente.

In questo scenario, viene anche illustrato come il sistema gestisce la **memoria dei messaggi**: ogni nuovo messaggio viene salvato in GroqService per mantenere il contesto delle interazioni successive, garantendo coerenza e continuità nelle descrizioni prodotte.

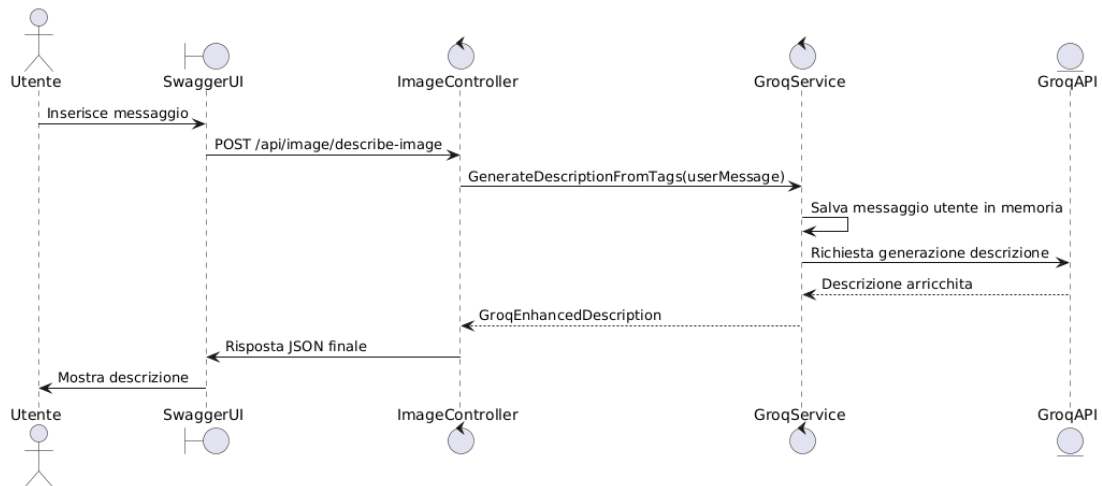


Figura 3 - Primo Diagramma di Sequenza (Utente inserisce un messaggio)

Nel secondo diagramma, in **Figura 4**, viene illustrato un esempio di come si comporta il sistema quando in un primo momento viene inserita un'immagine, nel nostro caso *"Gioconda.jpg"*, e successivamente viene cambiata con un'altra immagine, ovvero *"Colosseo.jpg"*.

Il diagramma mette in evidenza come, al caricamento della nuova immagine, **GroqService rilevi il cambiamento e azzeri automaticamente la memoria dei messaggi**, in modo da evitare che informazioni precedenti vengano applicate erroneamente al nuovo contesto visivo.

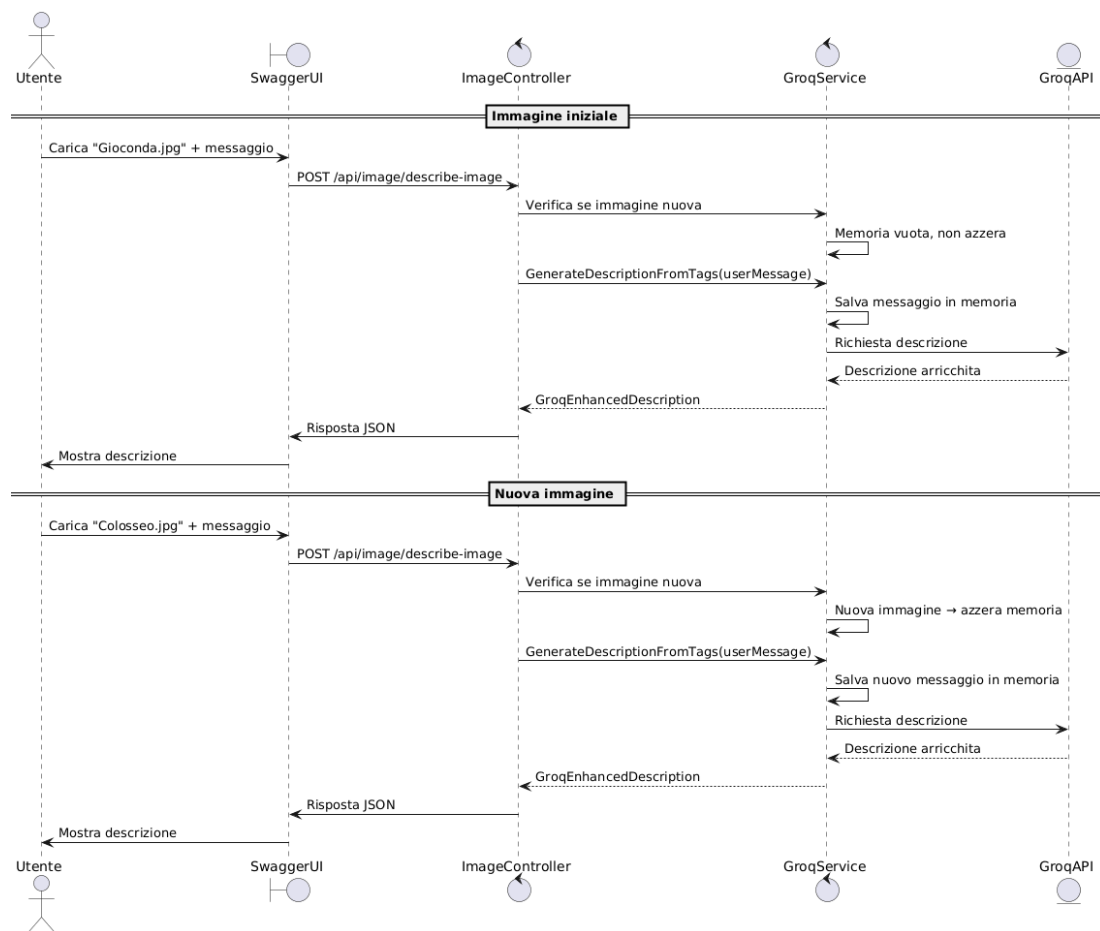


Figura 4 - Secondo Diagramma di Sequenza (Utente cambia immagine)

Capitolo 4: Funzionalità Aggiuntive

Per fornire un'esperienza migliore all'utente, sono stati introdotti diversi **controlli** che vanno a migliorare il progetto creato, rendendolo più affidabile e fluido.

4.1 Controllo sull'immagine

L'utente può caricare immagini di diverso tipo; tuttavia, alcune caratteristiche delle immagini possono comprometterne l'analisi da parte del sistema. Ad esempio, si preferisce non descrivere immagini dove presente un'ombra eccessiva, o sfocate.

Per questo motivo sono stati inseriti dei controlli volti a garantire la qualità dei contenuti.

In particolare:

1. **Controllo ombre e sfocature:** se tra i tag restituiti da Clarifai compare **[shadow]**, l'immagine viene considerata eccessivamente ombreggiata e viene scartata. In tal caso, viene mostrato un avviso all'utente, invitandolo a caricare un'immagine più adatta.
2. **Controllo risoluzione:** le immagini con risoluzione inferiore a 600x400 o superiore a 1920x1080 non vengono elaborate, mostrando il messaggio:
"Risoluzione non supportata (minima: 600x400, massima: 1920x1080)"
3. **Controllo formato file:** sono accettati solo file con estensioni JPG, JPEG, PNG, BMP e WEBP. In caso contrario, viene mostrato il messaggio:
"Formato immagine non supportato. Sono accettati solo i formati: JPG, JPEG, PNG, BMP, WEBP"
4. **Controllo assenza file:** se l'utente non carica alcuna immagine, viene visualizzato il messaggio:
"Nessun file immagine caricato"

4.2 Controllo Textfield

Qui è stato molto importante fare dei controlli su cosa inseriva l'utente, perché viene data anche la possibilità di non scrivere nulla. In particolare, è stato necessario considerare due casi:

1. **Messaggio nullo:** se l'utente non inserisce alcune testo viene inviato a Groq un messaggio *null*. In questo caso **non si creano problemi**. Semplicemente

viene detto al *prompt* di Groq, che se l'utente non inserisce nulla, devi descrivere l'immagine con le informazioni date dalle due API.

2. **Messaggio vuoto:** se l'utente inserisce solamente spazi, il messaggio, tecnicamente, non è più *null*, e viene interpretato come un input valido. In assenza di un controllo Groq, produceva descrizioni incoerenti e per qualche ragione, si verificavano perdite nella memoria dei messaggi inseriti precedentemente dall'utente.

Per prevenire il secondo problema, è stato necessario aggiungere un controllo sul messaggio inserito, ovvero:

“Il messaggio non può contenere solo spazi vuoti. Inserisci del testo significativo.”

4.3 Interfaccia utente personalizzata

Per migliorare l'esperienza utente, è stata sviluppata **un'interfaccia grafica personalizzata**, sostituendo Swagger UI.

Di seguito, viene illustrata in **Figura 5** la sezione di Swagger, in cui l'utente può inserire un'immagine, fornire una descrizione e avviare il programma, mentre la **Figura 6** mostra il risultato della descrizione generata dal sistema.

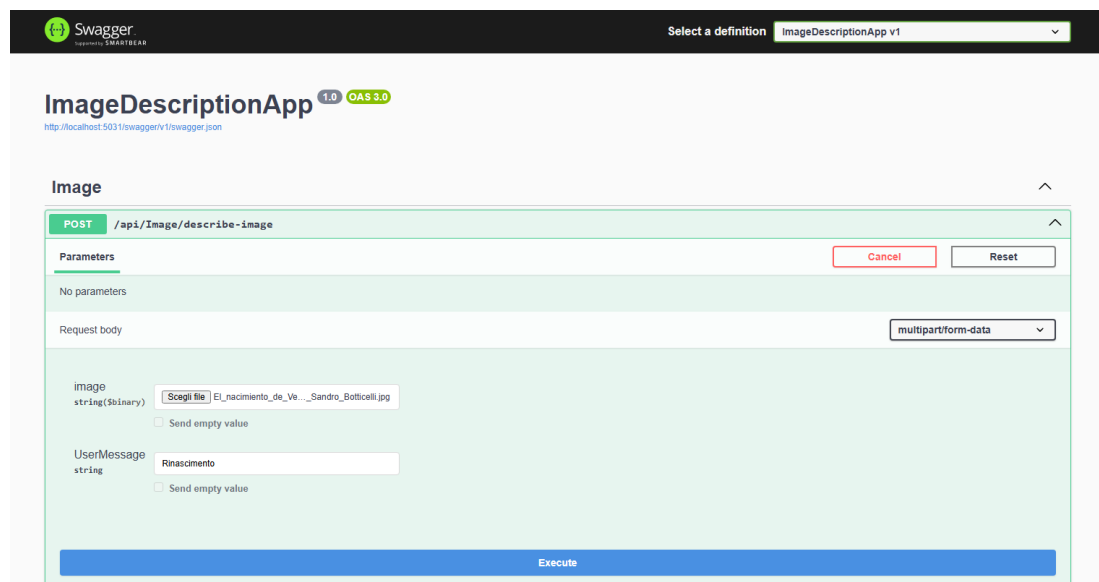


Figura 5 - Interazione con il sistema

l'immagine originale, valutando l'accuratezza della descrizione e individuando eventuali errori o imprecisioni. (Vedi **Figura 8**)

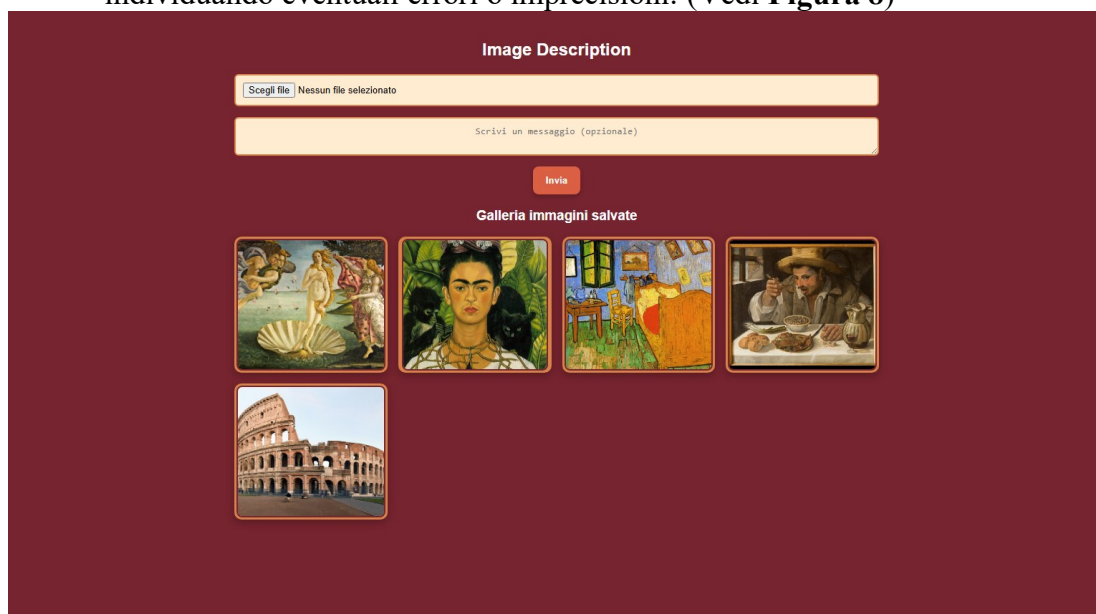


Figura 7 - Schermata di avvio

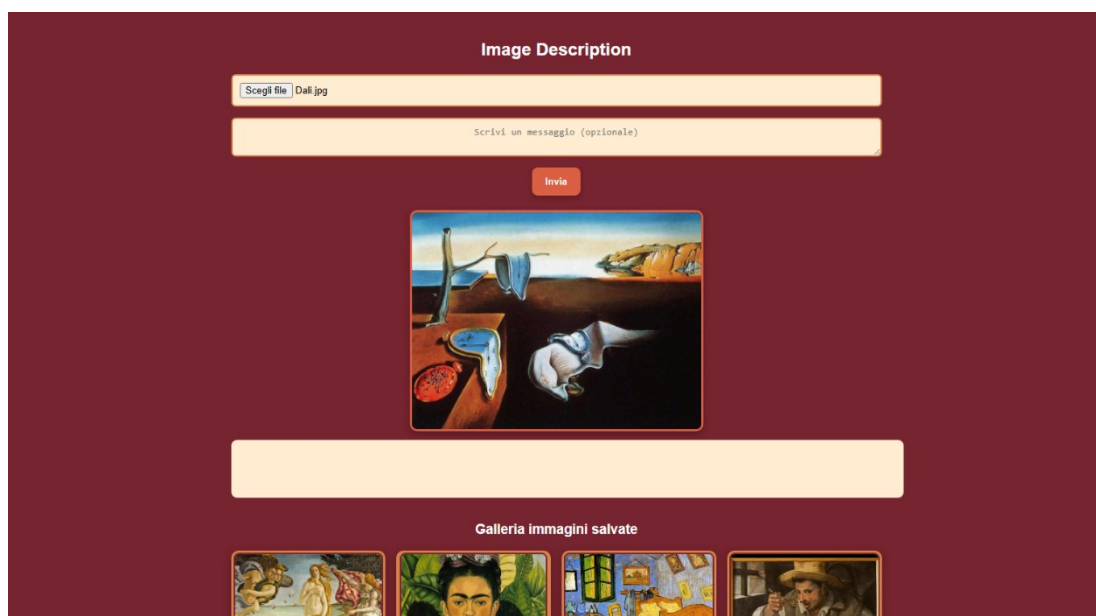


Figura 8 - Utente inserisce una foto

2. **Focus sul testo generato da Groq:** L'utente **visualizza principalmente il testo generato da Groq**, poiché con maggior interesse. Per questo motivo, è stato preferito dargli un ruolo centrale rispetto alle altre API. Ad ogni modo, viene data comunque la possibilità di visualizzare il contenuto delle altre API più il messaggio inserito dall'utente, per motivi legati alla chiarezza e completezza. A tal proposito, è stato aggiunto un apposito pulsante, dove,

premendolo compaiono le informazioni del caso. (Vedi **Figura 9** e **Figura 10**)

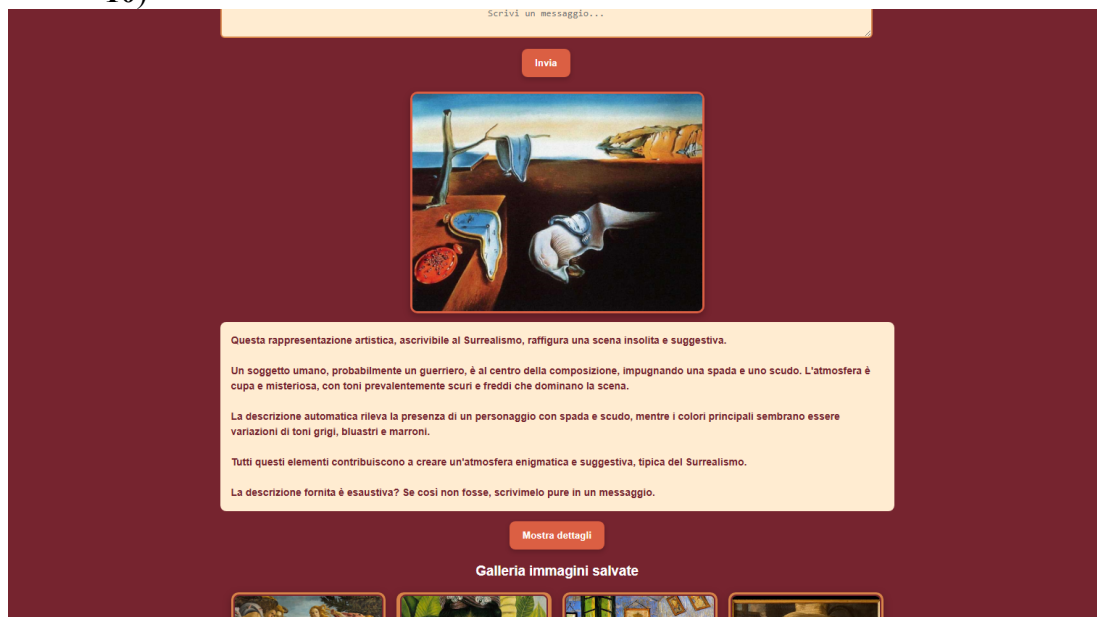


Figura 9 - Messaggio di Groq

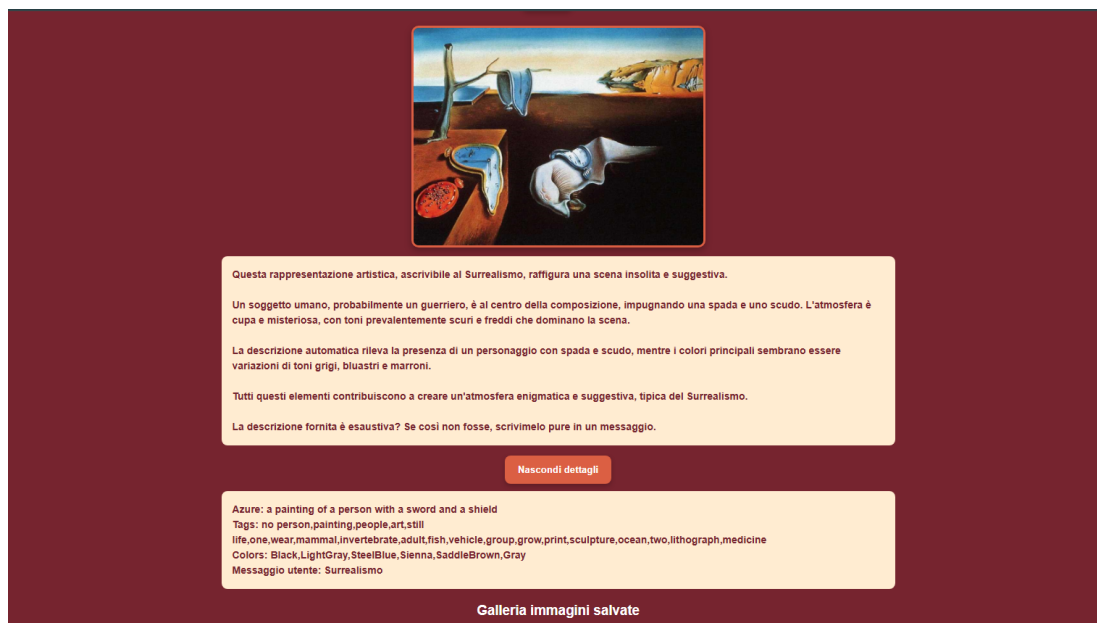


Figura 10 - Messaggi delle altre API + Messaggio Utente

3. **Storico delle immagini:** Le immagini precedentemente caricate, insieme alle relative descrizioni, vengono memorizzate in un database e **visualizzate nella parte inferiore dell'interfaccia**. Ciò è stato possibile grazie a un **database** (verrà meglio illustrato nel paragrafo seguente). che permette di

immagazzinare queste informazioni e darle all'utente. Questo storico compare nella parte di sotto della schermata, e da inoltre la possibilità all'utente, di modificare una descrizione di un'immagine descritta in passato. (Vedi **Figura 11**)

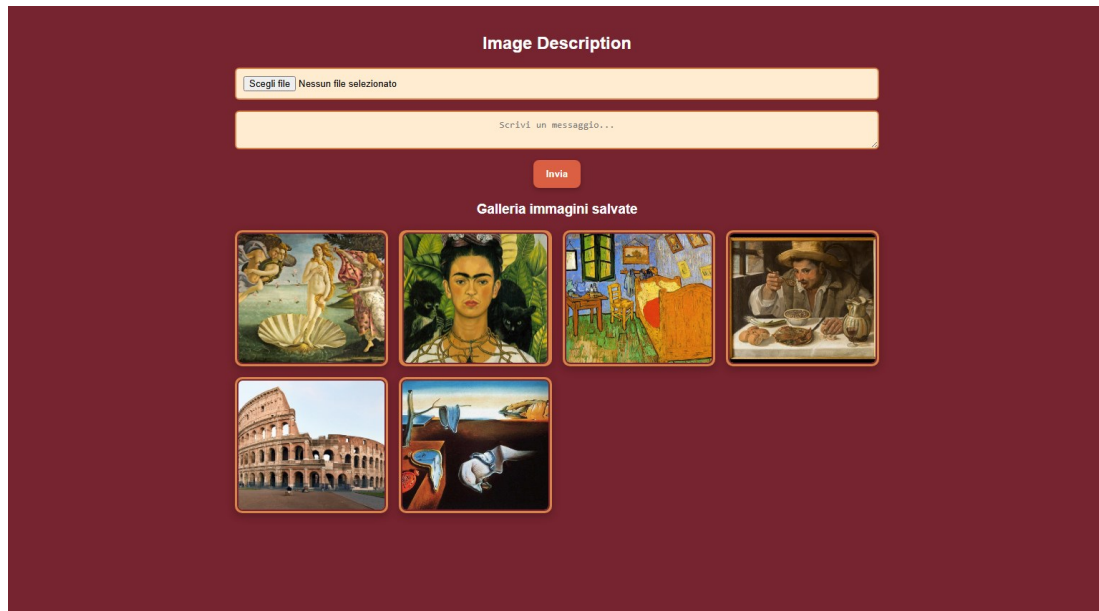


Figura 11 - Galleria Immagini Salvate

4.4 Database

Per garantire la persistenza e la gestione efficiente dei dati, soprattutto facilmente accessibile, è stato utilizzato un database relazionale.

La scelta è ricaduta su **SQLite**, perché l'implementazione è molto semplice: di fatto non richiede nessun'installazione di un server dedicato, poiché tutte le informazioni sono memorizzate all'interno di un unico file. Inoltre, la sua integrazione con .NET e con l'ambiente Visual Studio 2022 ha permesso di creare e gestire il database direttamente tramite terminale.

È stato scelto di utilizzare un modello relazione, ovvero una rappresentazione sottoforma di tabelle composte da righe (tuple) e colonne (attributi) per raccogliere i dati che servivano alla causa. Nel progetto ho definito solamente una tabella principale chiamata *Images*, che si occupa di gestire i file immagine e delle relative descrizioni date dal sistema intelligente.

Di seguito, ecco come è composta la tabella, in **Figura 12**, mostrando prima un'immagine del modello relazione, seguita dalla spiegazione di ogni attributo:

	Nome	Tipo di dati	Chiave primaria	Foreign key	Univoco	Controllo	Non NULL	Ordinamento	Generato	Valore predefinito
1	Id	INTEGER								NULL
2	AzureDescription	TEXT								NULL
3	ClarifaiColors	TEXT								NULL
4	ClarifaiTags	TEXT								NULL
5	ContentType	TEXT								NULL
6	CreatedAt	TEXT								CURRENT_TIMESTAMP
7	FileName	TEXT								NULL
8	GroqDescription	TEXT								NULL
9	ImageData	BLOB								NULL
10	ImageHash	TEXT								NULL
11	UpdatedAt	TEXT								CURRENT_TIMESTAMP
12	UserMessage	TEXT								NULL

Figura 12 - Modello relazionale del database: tabella “Images” con i campi

Definizione dei campi della tabella “Images”:

1. **Id:** chiave primaria univoca, auto-incrementale, utilizzata per identificare ciascuna immagine.
2. **AzureDescription:** descrizione fornita da Azure.
3. **ClarifaiTags/ClarifaiColors:** tags forniti da Clarifai.
4. **ContentType:** indica il formato dell’immagine.
5. **FileName:** nome del file immagine.
6. **GroqDescription:** descrizione generata da Groq.
7. **ImageData:** immagine salvata in formato binario.
8. **ImageHash:** hash dell’immagine, utilizzato per identificare univocamente i file e prevenire duplicati.
9. **CreatedAt / UpdatedAt:** timestamp per tracciare la creazione e l’ultima modifica del record.
10. **UserMessage:** messaggio fornito dall’utente.

Il database, quindi, si è rivelato un elemento importante all’interno del progetto, poiché non solo consente di organizzare e preservare i dati in maniera strutturata, ma contribuisce anche a migliorare significativamente la gestione e la fruizione delle informazioni da parte dell’utente.

Capitolo 5: Implementazione e descrizione delle funzionalità

In questo capitolo vengono illustrate le funzionalità interessanti che hanno reso possibile il corretto funzionamento del sistema. Vedremo il *prompt* di Groq nel dettaglio, **componente fondamentale**, e la sua evoluzione nel corso dello sviluppo. Per finire viene mostrato come viene gestita la memoria dei **messaggi inseriti** dall'utente di Groq.

5.1 Prompt di Groq

Il primo passo è stato normalizzare¹² la descrizione fornita da Azure, i tags di Clarifai, e il messaggio dell'utente, convertendoli in stringhe. I messaggi normalizzati prendono il nome di **{basicDescription}** per Azure, **{tags}** e **{colors}** per gli oggetti e i colori rilevati da Clarifai e **{combinedUserInfo}** il messaggio dell'utente.

Questa normalizzazione ha permesso di avere un formato uniforme, essenziale per costruire il prompt che guiderà Groq nella generazione delle descrizioni.

5.1.1 Prima versione

La prima versione del *prompt*, senza ancora aggiungere il messaggio inserito dall'utente. Si presentava in questo modo:

"Sei un assistente che riformula descrizioni di immagini in italiano in modo elegante, mantenendo fedelmente tutte le informazioni fornite. Le immagini raffigurano opere d'arte famose.

"Riscrivi questa descrizione in italiano in modo più scorrevole e naturale, senza aggiungere dettagli non presenti, ma sottolineando gli elementi unici e caratteristici dell'opera, come composizione, colori, personaggi e stile pittorico, autore e nome dell'opera: {basicDescription}"

In questa fase, il sistema era focalizzato principalmente sulle opere d'arte. Va notato che vengono date informazioni su come deve scrivere, in che lingua, e cosa deve trattare. Inoltre, si tentava di far indovinare il quadro semplicemente con Azure, ma solamente con una piccola descrizione sbagliava sempre il quadro.

¹²

In questo caso si intende la **normalizzazione** come un'operazione di un'informazione e trasformazione dei dati eterogenei provenienti da servizi diversi (Azure Computer Vision e Clarifai) in una rappresentazione testuale coerente.

5.1.2 Seconda versione

Con l'integrazione di Clarifai, il *prompt* è stato aggiornato. La nuova versione era strutturata come segue:

*"Hai a disposizione una serie di tag visivi e colori rilevati da un sistema di image recognition.
Tag rilevati: {tags}.
Colori principali: {colors}.
Inoltre, hai a disposizione una descrizione di base dell'immagine:
{basicDescription}.*

*Descrivi l'immagine in modo oggettivo, senza aggiungere dettagli inventati o aggettivi inutili e basandoti sulle risorse che ti ho indicato.
Deduci se possibile l'oggetto o gli oggetti presenti nell'immagine, il colore di ciascun oggetto della foto e le caratteristiche visive più evidenti.
Inoltre, se possibile, definisci qual'è il contesto principale dell'immagine.
La descrizione deve essere chiara e sintetica, di 3 righe.
Quando descrivi non menzionare i tag o i colori da cui hai prelevato le informazioni, ma cerca di produrre una descrizione fluida e naturale basata su di essi."*

In questa fase, Groq riceve tutte le informazioni rilevate dalle due API e poi gli viene detto cosa e come farlo. È stato specificato di non menzionare i tag, per offrire all'utente un'esperienza migliore tentando di rendere la descrizione quanto più umana possibile. Inoltre, da come si può leggere, la descrizione non prende come protagonista specifico un'opera d'arte, ma resta molto generale, così da poter descrivere qualsiasi immagine inserisca l'utente.

5.1.3 Versione definitiva

Con l'aggiunta, fondamentale, del messaggio inserito dall'utente, il prompt di Groq è stato **ulteriormente perfezionato**, perché doveva comportarsi in maniera diversa in base alle informazioni che arrivavano. Per realizzare ciò, sono state create delle istruzioni che deve seguire fedelmente in base agli eventi, e tutto al più, deve tentare di capire, solo se ha abbastanza informazioni, se l'immagine inserita è di famosa e risaputa. Ecco come si mostra il *prompt* definitivo:

*" Hai a disposizione una serie di informazioni per descrivere un'immagine.
- Tag rilevati da Clarifai: {tags}.
- Colori principali individuati: {colors}.
- Descrizione automatica da Azure: {basicDescription}.
{(hasUserMessage ? \$"- Informazioni certe dall'utente: {combinedUserInfo}." : "")}*

Importante: includi sempre nella descrizione i messaggi inseriti dall'utente. Se l'utente non inserisce nessun messaggio, continua con la descrizione senza dire che l'utente non ha inserito alcun messaggio.

Istruzioni:

1. Non introdurre mai la descrizione con frasi come “Non avendo informazioni” o “Proverò a descrivere...”. Inizia direttamente con la descrizione visiva, oggettiva, naturale e fluida, come se fosse parte di un catalogo o una guida illustrata.
2. Se l'utente non fornisce alcuna informazione aggiuntiva, descrivi ciò che si vede in modo diretto, senza elenchi, senza citare colori o tag letteralmente (es. “Tan”, “Gray”, ecc.).
Usa i tag e i colori solo per ispirare la descrizione, non per riportarli testualmente.
3. Cerca di dedurre di che colore sono gli elementi presenti nell'immagine sulla base delle informazioni che hai a disposizione.
4. Evita di aggiungere aggettivi inutili ai fini descrittivi, come 'quest'affascinante abitazione' o 'questo elegante mobile'. Concentrati su ciò che è visibile e rilevante per l'immagine.
Scrivi una descrizione elegante, raffinata e fluida (in stile da catalogo).
- Se l'utente ha fornito solo alcune informazioni (es. solo il titolo, il nome dell'autore o solo le dimensioni), cerca comunque di dedurre il resto se possibile, oppure indica solo ciò che è certo.
Esempio:
Dimensioni: 80x120 cm
5. Concludi sempre con:
La descrizione fornita è esaustiva? Se così non fosse, scrivimelo pure in un messaggio.

Scrivi tutto in italiano, senza ripetizioni né linguaggio artificiale."

Prima di generare la descrizione, il sistema verifica se l'utente ha fornito un **messaggio significativo**, grazie alla seguente riga di codice:

```
"var hasUserMessage = !string.IsNullOrEmpty(combinedUserInfo);"
```

string.IsNullOrEmpty(...) restituisce **true** sia per **null** sia per **stringhe** che contengono solo spazi/tab/ritorni a capo.

Da come si può leggere nel *prompt*, viene specificato che il messaggio dell'utente ha un peso maggiore rispetto alle altre informazioni e che man mano che sono stati eseguiti dei test sono stati fatti degli accorgimenti, come l'istruzione che dice di evitare aggettivi ai fini descrittivi, con l'intenzione di migliorare il quanto più possibile l'esperienza dell'utente.

5.2 Creazione Memoria Groq

Come spiegato nei capitoli precedenti, **era fondamentale** non perdere informazioni, soprattutto informazioni con un peso elevato, quindi è stato necessario creare una memoria, che si azzerasse se all’inserimento di una nuova immagine.

All’inizio, si è provato a dire a Groq di salvare le informazioni che inviava l’utente, ma non ha funzionato, quindi bisognava creare una **collezione di oggetti**¹³.

La scelta è ricaduta sulla **lista ordinata**, ovvero una collezione ordinata di elementi in cui è possibile inserire, rimuovere e accedere agli elementi tramite la loro posizione (indice).

5.2.1 Metodi Creati

Per la realizzazione di questa funzione sono stati creati tre metodi¹⁴.

1. **ResetHistory:** resetta la memoria di Groq, semplicemente rimuovendo gli elementi salvati nella lista. Ecco come si presenta:

```
“ public void ResetHistory() { _userMessagesHistory.Clear(); } “
```

2. **UpdateImageHash:** rileva cambiamenti dell’immagine, poiché se presente un’immagine nuova rispetto a quella precedente, bisogna pulire la lista. Di seguito viene mostrato il codice:

```
“public void UpdateImageHash(byte[] imageBytes) {  
    using var sha256 = SHA256.Create();  
    var hash = Convert.ToBase64String(sha256.ComputeHash(imageBytes));  
  
    if (_lastImageHash != hash) {  
        _lastImageHash = hash;  
        ResetHistory();  
    } }”
```

L’idea alla base è stata quella di **convertire l’immagine in un codice univoco** e poi fare un confronto con il codice precedente con quello nuovo. Per realizzare questa idea ho dovuto utilizzare l’algoritmo **SHA256**, che produce

¹³ In informatica, una **collezione di oggetti** è un insieme di elementi (oggetti) organizzati in una struttura dati, che permette di **archiviare, gestire e accedere** agli oggetti in modo ordinato e coerente

¹⁴ In informatica, un **metodo** è un **blocco di codice che esegue una specifica funzione o operazione**.

un hash¹⁵ univoco. Fatto ciò, per facilitare il confronto, ho dovuto convertire l'hash in una stringa con **Base64**¹⁶. Dopodiché viene fatta una verifica, ovvero se l'hash appena calcolato è diverso da quello precedente, viene resettata la cronologia, chiamando il metodo **ResetHistory()**.

3. **AddUserMessageToHistory**: serve a gestire lo storico dei messaggi inseriti dall'utente. Il codice è come segue:

```
“public void AddUserMessageToHistory(string message) {  
    var trimmed = message.Trim();  
  
    if (!string.IsNullOrEmpty(trimmed) &&  
        !_userMessagesHistory.Contains(trimmed, StringComparer.OrdinalIgnoreCase))  
        _userMessagesHistory.Add(trimmed); }
```

I messaggi prima di essere inseriti nella lista eseguono due funzioni:

1. Grazie alla funzione **Trim()** vengono rimossi spazi bianchi iniziali e finali dal messaggio. Questo serve a evitare che messaggi uguali, ma con spazi diversi all'inizio o alla fine, vengano considerati distinti.
2. Verifica che il messaggio non sia già presente nello storico e che contenga testo significativo.

¹⁵ Un **hash** (SHA-256) è il risultato di una funzione crittografica che trasforma un input di qualsiasi dimensione (come un'immagine o un testo) in una stringa di lunghezza fissa. I caratteri variano in base all'immagine. Un'immagine ha sempre lo stesso codice.

¹⁶ **Base64** è un metodo di codifica che trasforma dati binari (come un array di byte) in **testo leggibile** costituito da lettere, numeri e simboli.

Capitolo 6: Demo

Dopo aver parlato di come funziona il sistema intelligente creato, in questo capitolo si vuole far vedere **una piccola dimostrazione pratica**.

L'immagine scelta per la dimostrazione è *La Venere di Botticelli*, un dipinto famoso ricco di dettagli. In **Figura 13**, viene rappresentata l'immagine scelta, in modo da rendere più agevole il confronto con la descrizione generata.

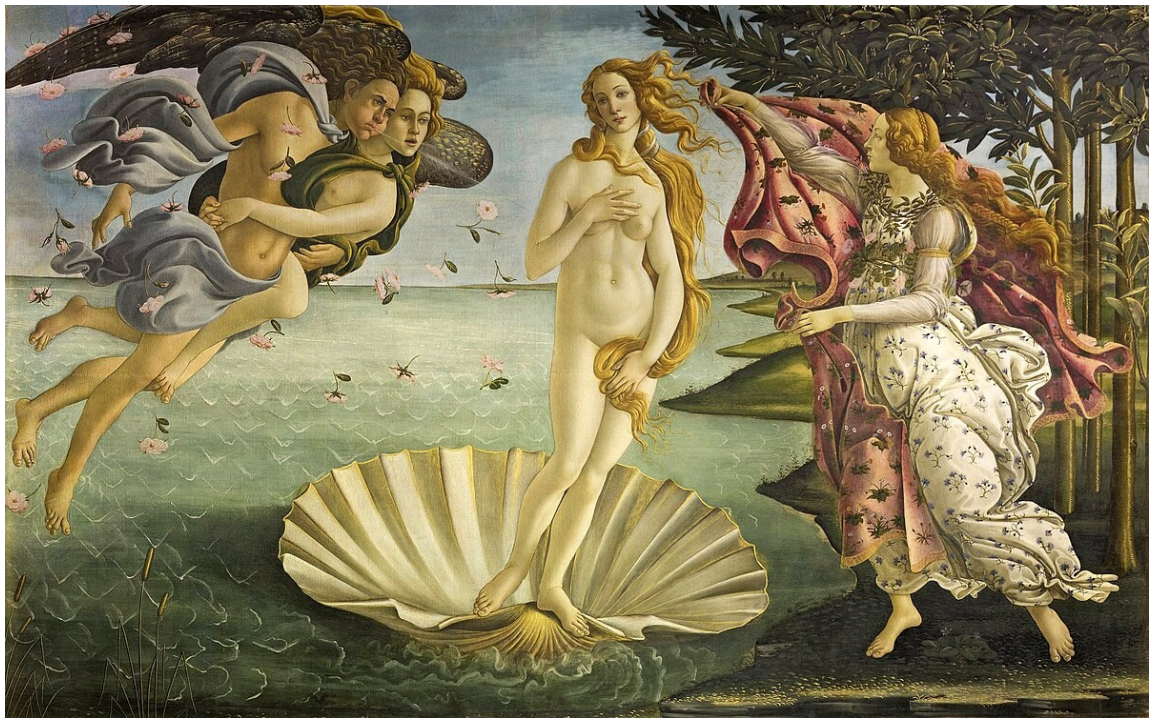


Figura 13 - Sandro Botticelli, *La Nascita di Venere* (1485 ca.)

Inizialmente, è stato preferito **non dare informazioni al sistema intelligente**, quindi non è stato scritto nulla all'interno del textfield e questo è stato il risultato:

"Questa illustrazione raffigura un gruppo di persone in un'atmosfera legata alla religione e alla mitologia. Al centro della scena, una donna è rappresentata in mezzo a un gruppo di figure, probabilmente in un contesto narrativo legato a una storia antica o a un credo religioso. Intorno a lei, si possono vedere elementi che richiamano un tempio o un luogo sacro, decorato con dettagli dorati e scolpiti. La scena è dominata da toni caldi e terrosi, con sfumature più scure che evocano un senso di profondità e storia. Le figure appaiono inserite in un ambiente riccamente ornato, con una forte presenza di elementi simbolici e narrativi.

La descrizione fornita è esaustiva? Se così non fosse, scrivimelo pure in un messaggio."

Da come si può leggere, il sistema intelligente ha visualizzato delle persone, soprattutto ha capito che al centro dell'immagine viene rappresentata una donna e ha fatto delle ipotesi sull'ambiente che le circonda. Per concludere, è riuscita a captare e descrivere i colori dell'immagine, definendoli caldi e terrosi. **Da non sottovalutare**, ha detto che l'immagine è **in profondità**. (anche gli spazi, tra un blocco di testo e l'altro, sono stati inseriti da Groq, per rendere quanto più fluida la lettura)

Adesso, è stata inserita la parola "Venere". Di seguito la descrizione prodotta:

"Questa rappresentazione artistica raffigura un gruppo di persone in un'atmosfera mitologica e religiosa. Al centro della scena, una figura femminile imponente, che potrebbe essere interpretata come Venere, emerge con una presenza dominante. La sua pelle ha un tono caldo e dorato, mentre i suoi capelli e il pannello che la avvolge mostrano riflessi più scuri e profondi.

Intorno a lei, si notano figure maschili e forse altre rappresentazioni divine o mitologiche, inserite in un contesto che sembra richiamare un tempio o un luogo di culto antico. Le figure sono rappresentate con stili che richiamano l'antichità, sia nella posa che nel trattamento dei dettagli.

La scena è caratterizzata da toni generali che vanno dal beige al grigio scuro, con tocchi più caldi e dorati che accentuano i contorni delle figure e degli elementi architettonici. L'insieme risulta in una rappresentazione suggestiva e ricca di dettagli, inserita in un contesto che sembra voler narrare una storia mitologica o religiosa.

La descrizione fornita è esaustiva? Se così non fosse, scrivimelo pure in un messaggio."

Il sistema ha intuito chi potrebbe essere il protagonista del quadro, dicendo che la Venere si trova al centro della figura, inoltre, aggiunge ancora più dettagli su di essa, come il tono della sua carnagione e i suoi capelli. Rispetto a prima, ha definito anche il sesso maschile delle persone di fianco alla Venere.

Per concludere, è stata voluta aggiungere un'altra informazione, ma non sull'immagine, ma su chi ha dipinto questo quadro, ovvero Sandro Botticelli e questo è stato il risultato:

"Quest'opera d'arte raffigura una scena mitologica e religiosa, ispirata al Rinascimento italiano.

La figura centrale è quella di una donna, rappresentata con tratti eleganti e sereni, vestita con un abito fluente. Il suo volto è caratterizzato da lineamenti delicati e da un'espressione dolce e serena.

Intorno a lei, si notano varie figure, tra cui un uomo, rappresentato come un eroe o un santo, con un'aura di luce dorata intorno al capo.

Sul fondo, si intravede un ambiente architettonico, con elementi che ricordano un tempio o una struttura antica.

L'intera scena è caratterizzata da toni caldi e pastello, con sfumature di marroni e grigi che creano un'atmosfera raccolta e contemplativa.

L'opera, che prende il nome di La Nascita di Venere, è chiaramente ispirata alla mitologia classica e alle credenze religiose dell'antichità, e presenta elementi simbolici e narrativi.

L'autore dell'opera è Sandro Botticelli, e la rappresentazione è quella di Venere, dea della bellezza e dell'amore nella mitologia romana.

La descrizione fornita è esaustiva? Se così non fosse, scrivimelo pure in un messaggio."

Questo è il risultato finale. Il sistema intelligente, grazie all'integrazione delle diverse API e all'interazione con l'utente, è stato in grado di riconoscere e descrivere correttamente l'immagine fornita. Come si può osservare, il modello identifica il **nome del quadro**, e, a partire da tale informazione, diventa possibile reperire e restituire **tutti i dettagli pertinenti sull'opera**.

Considerata come una descrizione soddisfacente, non si è ritenuto necessario condurre ulteriori test.

Capitolo 7: Conclusioni e prospettive future

In questo capitolo si vogliono **tirare le somme** di quella che è stato il progetto sviluppato, e dare delle **possibili continuazioni** di questo progetto.

Il lavoro svolto ha quindi permesso di realizzare un sistema in grado di catturare un'immagine, interagire personalmente con esso, dare una descrizione automatica di questa e infine, memorizzarla in un database. Si è partiti da una descrizione minimale scritta da **Microsoft Azure** fino ad arrivare, grazie all'aiuto di **Clarifai API**, il messaggio utente, ma soprattutto **Groq**, che prendendo in input tutte le informazioni è riuscito a dare una descrizione precisa ed efficiente come abbiamo illustrato nel capitolo precedente. È stata mostrata l'importanza del messaggio inserito dall'utente, e come questa cambi drasticamente la descrizione dell'immagine.

Inoltre, per facilitare l'uso e offrire un'esperienza quanto più semplice ed efficace, sono state modificate l'**interfaccia utente**, e un database in **SQLite** che permette la memorizzazione di immagini e descrizioni.

Tuttavia, la vera sfida **non è stata solamente tecnica**, quindi implementazione e ricerche in internet, ma anche quella legata alle risorse disponibili. L'intero sistema, infatti, è stato creato facendo affidamento ad API **completamente gratuite**, comportando quindi un lavoro aggiuntivo di ricerca, adattamento e integrazione tra di loro.

Nel corso del progetto sono state diverse le soluzioni prese in considerazione, ma non tutte sono state possibili da realizzare, chi per motivi di tempo chi per motivi legati alla complessità. Tuttavia, **meritano di essere sviluppate nel futuro**, poiché andrebbero a ottimizzare l'esperienza da parte dell'utente.

Tra le possibili evoluzioni si individuano:

- Rendere l'interazione con il sistema **una vera e propria chat**, come siamo abituati ad oggi con i sistemi di AI che si trovano in circolazione. Questa implementazione porterebbe l'esperienza su un altro livello, dato che il textfield risulta leggermente meno immediato, ma comunque funzionale. A questo punto si possono fornire, non solo delle informazioni, **ma fare delle vere e proprie domande**.

Una volta inserite tutte le informazioni, e il sistema ha capito l'immagine, non serve più inviare informazioni, ma sarebbe meglio fare delle domande, ottenendo così ulteriori chiarimenti.

- Creare **un'esperienza personalizzata per il singolo utente**. Per fare ciò, bisognerebbe far creare un account all'utente tramite una pagina di registrazione, per poi fargli fare il login. Così facendo, ogni utente ha le proprie immagini con relativa descrizione salvate all'interno di un database

comune, modificandolo opportunamente. Bisognerebbe aggiungere una tabella per gli utenti, dove viene popolata dalle informazioni date dall'utente quando compila il form di registrazione.

Se ciò dovesse essere sviluppato, bisogna prestare attenzione a non salvare **le password in chiaro, ma bensì, dovranno essere crittografate.**

- Aggiungere un'altra schermata dove vengono **raccolte tutte le immagini con descrizione**. Nel progetto attuale, tutto quanto viene visualizzato in una sola schermata, il che se ci sono poche informazioni salvate nel database risulta accettabile, ma se aumentano la schermata perde di qualità.

Quindi sarebbe opportuno creare un **catalogo** d'immagini e quando si clicca su di essa si può scegliere se visualizzare la descrizione o modificare il contenuto.

Indice delle Figure

Figura 1- Diagramma dei casi d'uso	12
Figura 2 - Diagramma delle Classi.....	14
Figura 3 - Primo Diagramma di Sequenza (Utente inserisce un messaggio)	23
Figura 4 - Secondo Diagramma di Sequenza (Utente cambia immagine)	24
Figura 5 - Interazione con il sistema	26
Figura 6 - Descrizione generata dell'immagine.....	27
Figura 7 - Schermata di avvio.....	28
Figura 8 - Utente inserisce una foto	28
Figura 9 - Messaggio di Groq	29
Figura 10 - Messaggi delle altre API + Messaggio Utente.....	29
Figura 11 - Galleria Immagini Salvate	30
Figura 12 - Modello relazionale del database: tabella "Images" con i campi	31
Figura 13 - Sandro Botticelli, La Nascita di Venere (1485 ca.).....	37